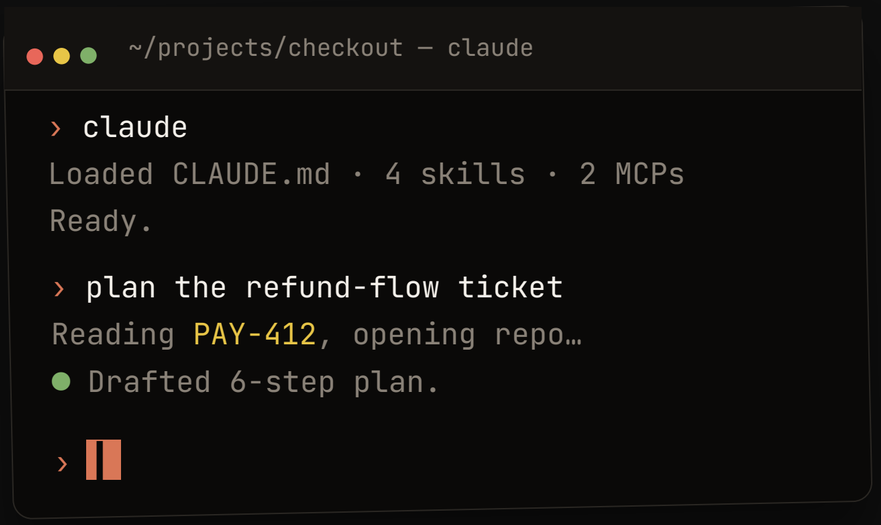
AN INTERNAL WALKTHROUGH

Claude Code.

How we use the CLI to plan, build, and ship — and the small habits that make it actually pay off.



CONTEXT

Why use Claude CLI?

01 — EXPECTATION

Clients expect AI-first workflows.

Increased productivity isn't a perk anymore — it's the baseline they're paying for.

02 — FOCUS

Automate the repetitive work.

Hand off the toil so you can spend your attention on the parts that need a human.

03 — HEADROOM

More time for planning & design.

When implementation gets cheaper, the leverage shifts upstream into thinking.

COMPARE

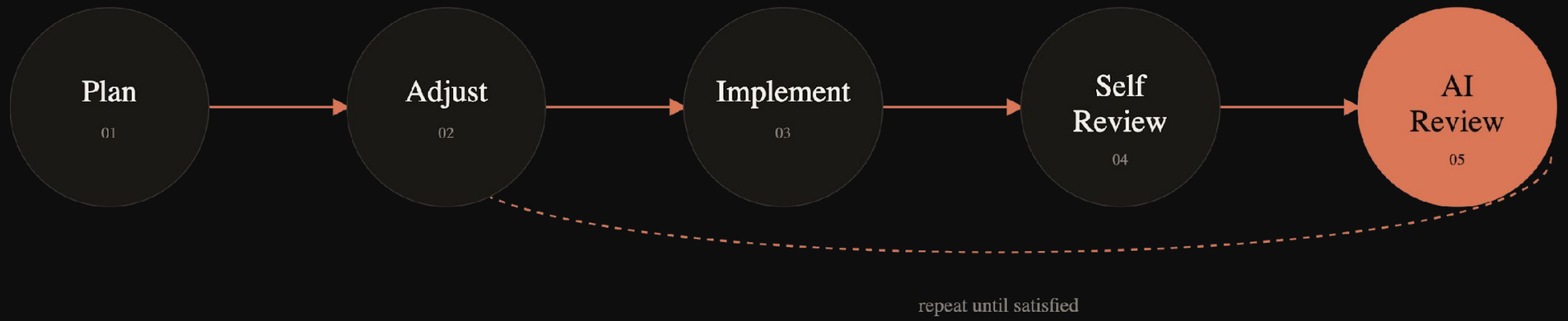
Claude Web vs Claude Code

	CHAT (WEB)	CLI (CODE)
Context	Conversation + uploaded project files	Direct access to repo + local filesystem
Workflow	Copy / paste from chats	Reads & writes directly into your project
Shell	N/A	Runs shell commands to debug, test, ship
Use case	Drafting docs, exploring new topics	Codebase planning, implementation, debugging

METHOD

Core workflow

Five steps. The last four loop until the diff feels right.



Steps 02 → 05 form the inner loop. Don't be afraid to discard the diff and re-plan when something feels off — cheaper than fighting a bad direction.

PART ONE

Talking to Claude.

Prompting, picking the right model, and the demo that makes it click.

SKILL

Prompt engineering

State the task , the constraints , and the output format you want.

Reference exact files, functions, and tickets — don't make Claude guess.

Break long requests into numbered steps.

If the answer is wrong during planning, refine it by prompting further — don't jump to implementation.

vague

```
› fix the checkout bug
```

specific

```
› In checkout/refund.ts , refunds over $500 fail silently. (1) reproduce with the test in refund.test.ts, (2) propose a fix, (3) wait for my approval before editing.
```

CHOICE

Model selection

Most tasks run fine on Sonnet. Reach for Opus only when planning large features or chasing bugs the smaller models couldn't crack.

HEAVY

OPUS

Deep reasoning

Complex refactors. Architecture work. Debug sessions where lower tiers stall out.

SONNET · DEFAULT

Daily coding

Mid-size refactors, PR review, most feature work. Your everyday driver.

HAIKU

Quick edits

Formatting, batch text changes, small chores you'd otherwise do by hand.

Effort level is also tunable. `xHigh` (the default) is recommended for now — revisit as the models change.

LIVE DEMO

Ticket workflow.

Pick a real ticket. Plan it together, then watch Claude turn the plan into a diff.

01 Read the ticket together

02 Draft the plan in `/plan` mode

03 Implement, then ask Claude to review its own diff

04 Open the PR with `gh pr create`

PART TWO

Configuring Claude.

CLAUDE.md, skills, custom commands, plugins, hooks — the wiring that turns Claude into a teammate that knows your project.

CONTEXT FILE

CLAUDE.md

Loaded into every session. The single source of truth Claude reads first.

WHAT GOES IN IT

- Project stack & major library versions
- Project structure
- Lint, test, and build commands
- Project-specific conventions

RULE

Keep it up to date.

Don't let actual implementation drift from the documentation — stale CLAUDE.md is worse than none.

CLAUDE.md

```
# Checkout Service Stack: Node 20 · TS 5.4 · Postgres 16 Framework: Fastify 4 · Drizzle ORM ## Commands pnpm dev · run locally pnpm test · vitest watch pnpm lint · biome --write ## Conventions - Throw DomainError , never raw Error - No raw SQL outside db/queries/
```

REUSABLE WORKFLOWS

Skills

Auto-loaded by Claude. Define a workflow once; let Claude invoke it whenever the situation matches.

USE THEM TO AUTOMATE REPETITIVE TASKS

RELEASE NOTES

Generate release notes

From the diff between two tags, in your team's voice, in the format you actually publish.

SCAFFOLDING

New controller, your way

Following your project's conventions for routing, validation, error handling, and tests.

POST-DEPLOY

Sentry / AppSignal sweep

Auto-scan for new issues after each release and surface what looks regression-shaped.

ON-DEMAND

Custom commands

Like skills — but you invoke them, when you want them.

01 — ADD

Drop a markdown file at `.claude/commands/<name>.md`

02 — INVOKE

Call it from the chat: `/<name>`

.claude/commands/review-pr.md

```
# /review-pr
Review the current branch's diff against
main. Check for:

- missing tests for new logic
- N+1 queries in db/queries/
- breaking API changes
- TODOs or console.logs

Output a checklist of findings, grouped
by severity.
```

```
› /review-pr
```

BUNDLES OF SKILLS + COMMANDS

Plugins

Add ready-made skills and commands you can pull straight into your workflow.

PLUGIN
Brainstorming

PLUGIN
Planning

PLUGIN
Debugging

PLUGIN
Code review

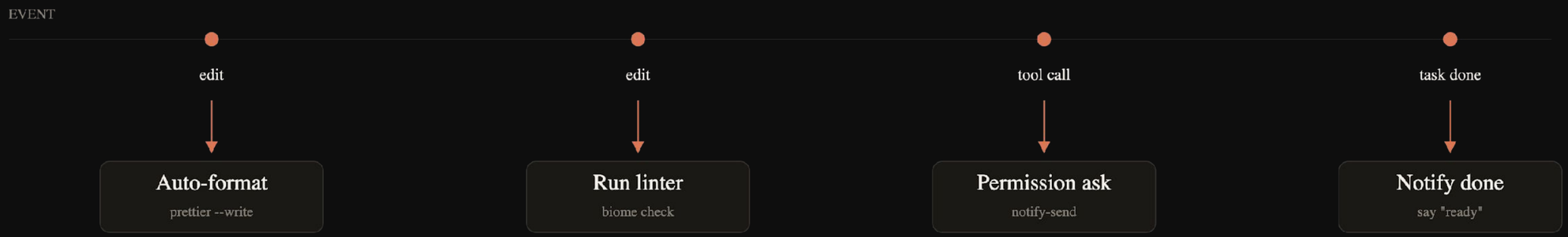
TIP
Big features?
Reach for the `TDD` and `Subagent driven development` skills to actually execute the plan.

DISCOVER
Browse the marketplace
There's probably already a plugin for that thing you keep meaning to script.

ALWAYS-ON

Hooks

Shell commands Claude runs automatically on different event types. Use these for steps that must *always* run.



PART THREE

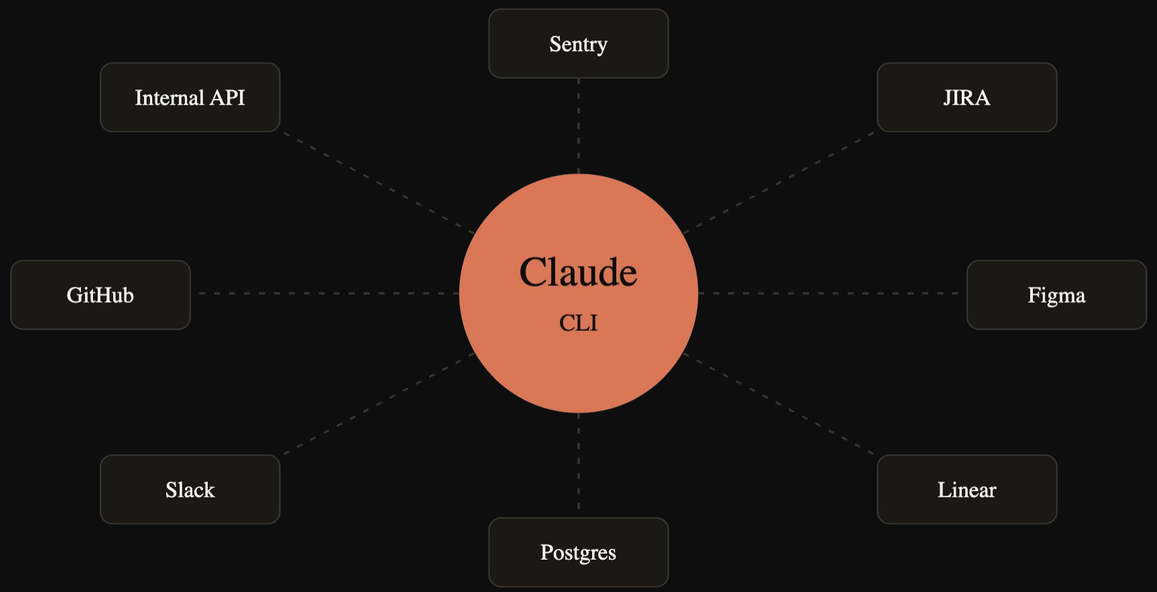
Extending Claude.

MCPs, browsers, CLI tools, subagents — give Claude hands and let it reach beyond the chat window.

BRIDGES

MCPs

Let the CLI talk to external tools — Sentry, JIRA, Figma, your own internal services.



SENTRY MCP

Fetch issues directly into context — Claude can triage and propose fixes inline.

JIRA MCP

Pull tickets into the chat without copy/paste. Acceptance criteria become real planning input.

FIGMA MCP

Read designs and compare to current implementation.

CAVEAT

Many MCPs **bloat the context window** . Enable per task — not all at once.

EYES ON THE UI

Browser tools

Claude Code can drive a real browser to test UI changes autonomously — built-in, or via the Playwright MCP, the `/chrome` plugin, or browsers agents.

VISUAL VERIFY

See its own changes

Let the AI check a UI change visually and iterate without your input.

SMOKE TEST

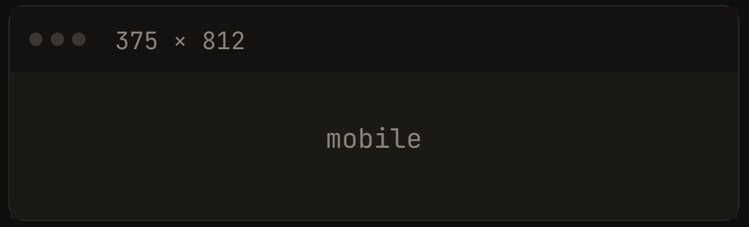
Post-fix walkthroughs

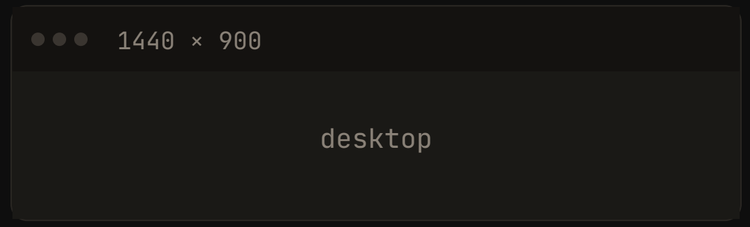
After a bug fix or new feature, run a quick scripted walk-through of the happy path.

RESPONSIVE

Layouts at every width

Snapshot at mobile, tablet, desktop — catch the breakpoint that broke.





YOUR SHELL, ITS HANDS

CLI tools

Claude has access to your shell — it can use any CLI tool you've already installed.

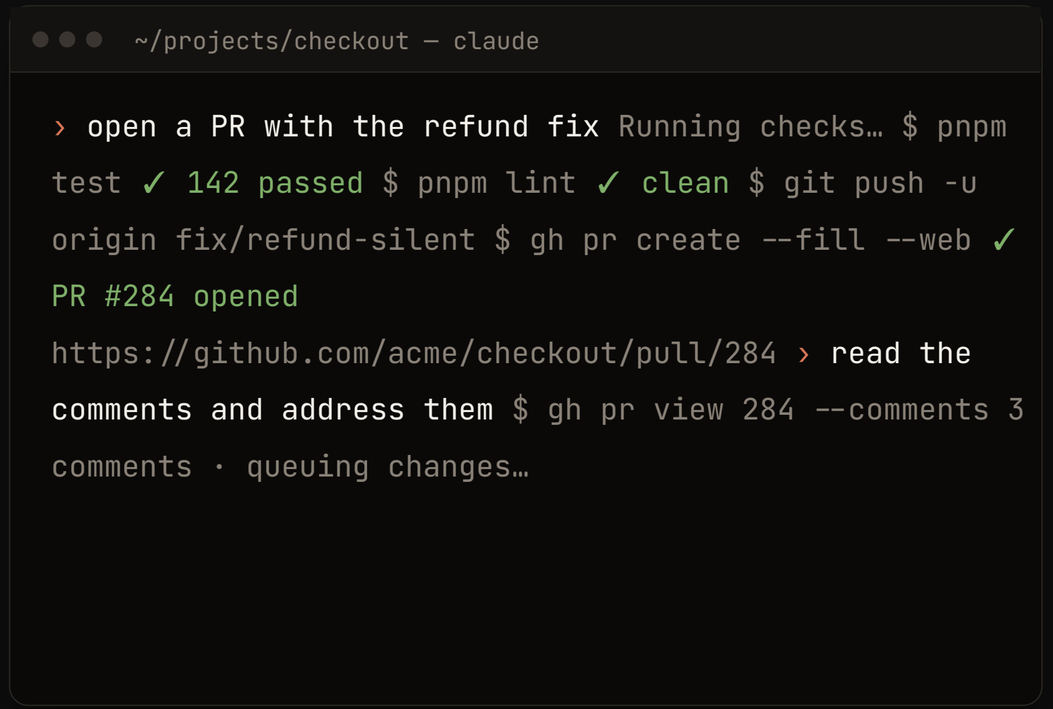
EXAMPLE · GITHUB CLI

Create PRs from the command line

Draft PR descriptions from the real diff

Read PR comments and action them

Same idea applies to any tool you live in — `docker`, `kubectl`, `aws`, `stripe` , your own scripts.



PARALLELISM

Subagents & worktrees

SUB-AGENT

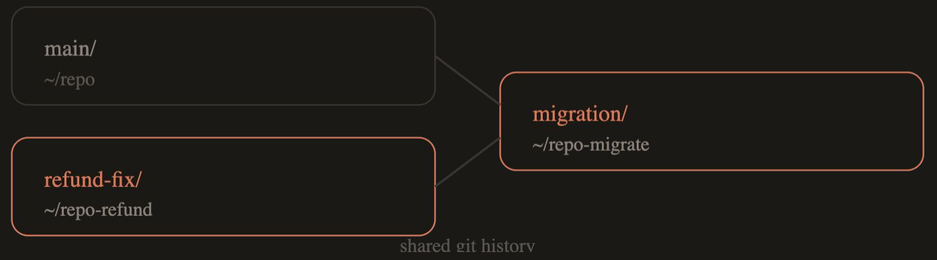
A separate task with its own context

Run independent tasks in parallel where only the output matters — e.g. exploring the codebase during planning. Claude spins them up automatically for most tasks; prompt explicitly when you want one.

WORKTREE

A separate working directory via git worktrees

Work on two parallel tasks at once. Use worktrees for long-running tasks that need minimal interaction after planning. Keep your main worktree for shorter tasks and for testing changes after the parallel ones complete.



PART FOUR

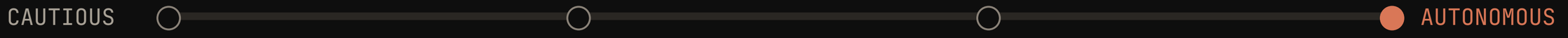
Operating safely.

Permissions, sessions, usage, insights — the operational habits that keep quality high and bills low.

TRUST DIAL

Permission modes

MODE	FUNCTIONALITY	USE CASE
Default	Asks before *every* action.	First sessions on a new repo. Sensitive code paths.
Plan	Read-only.	Planning a feature or fix before any implementation.
Accept Edits	Edits without asking; still prompts on shell commands.	Implementing an agreed plan, small bug fixes.
Auto	Local classifier accepts/rejects commands without input.	Advanced users on agentic workflows. Not recommended for day-to-day.



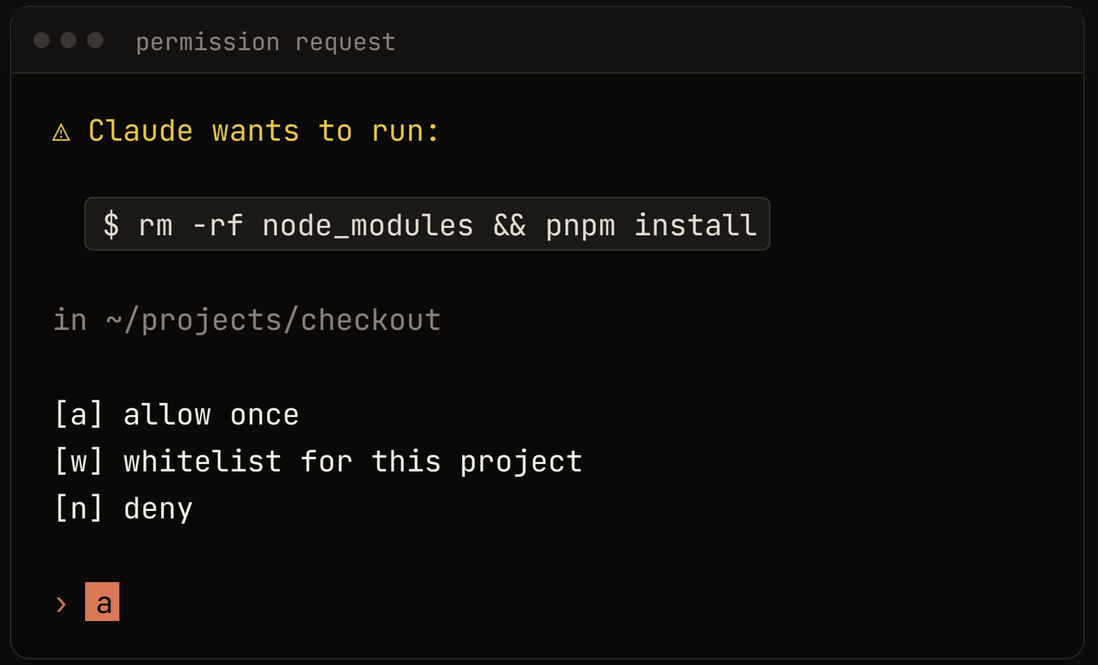
WHITELIST

Permissions

> Claude asks for permission before running any shell command.

Commands can be added to a per-project whitelist.

Be careful what you whitelist — make sure you understand what a command does before allowing it.



CONTEXT HYGIENE

Session management

Actively manage the context window. A bloated context drops output quality and runs up cost.

COMPACTION

Summarise & restart

Boil the chat down to a summary, then begin a fresh conversation with just the essentials.

long chat → summary

RESUME

Pick up where you left off

Continue a session from where you stopped. Use only when needed — resuming reloads the whole context and re-incurs the cost.

old session ↺ reloaded in full

TOKENS COST MONEY

Usage tracking

Be mindful of token usage.
Always start a new session for a new task.
If a session has dragged on with back-and-forth from a wrong turn, restart and improve your initial prompt to remove the ambiguity that caused it.

`/context` — what's eating context

See the breakdown of what's currently in the window.

`/usage` — limits & budget

Track usage limits — plan your work as you approach them.

`/statusline` — live in the prompt

Configure a live status line in your terminal so you don't have to ask.

SELF-COACHING

Claude Insights

Claude can help you improve your usage of the CLI. We don't always notice the tasks that could be automated, or the patterns that regularly cause friction.

monthly insights

```
› /insights Analysing 142 sessions over 28 days… ●
Repeating pattern detected You often ask Claude to write
Conventional Commit messages. → Consider a custom command.
● Friction /plan is followed by re-planning 38% of the
time. Try referencing more files up-front. ● Cost 14% of
tokens are re-loaded sessions.
```

CADENCE

Run it monthly

You need a large sample size before patterns become meaningful — once a month is plenty.

Use the `/insights` command to generate a report of your Claude usage and see if it suggests something you can incorporate into your workflows.

CLOSING

Best practices

The final output is the developer's responsibility — regardless of how the code itself was written.

01

Always review diffs before raising PRs.

02

Always test changes manually before moving tickets to QA.

03

Planning is where the engineering happens.

Make sure you understand the proposed solution before signing off on it.

04

Claude doesn't know your team.

It's not aware of project-specific conventions or blockers — verify against your team's standards.

Use the tool. Own the output.